

数据结构与算法 A

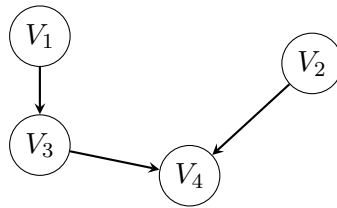
common-14 秋-期末

一、填空 (27 分)

1. 当各边上的权值 _____ 时，BFS 算法可用来解决单源最短路径问题。

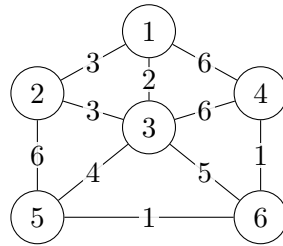
- (A) 均相等 (C) 不一定相等
(B) 均互不相等

2. 有向图 G 如下图所示：



- (1) 写出所有可能的拓扑序列：_____。
(2) 添加一条弧 _____ 之后，则仅有唯一的拓扑序列。

3. 设有如下所示无向图 G ，用 Prim 算法构造最小生成树所走过的边



的边集。

4. 如果只想得到 1000 个元素组成的序列中第 5 个最小元素之前的部分排序的序列，用 _____ 方法最快。

- (A) 起泡排序 (C) 堆排序
(B) 快速排序 (D) 简单选择排序

5. 若要求尽可能快地对序列进行稳定的排序，则应选 _____

- (A) 快速排序 (C) 冒泡排序
(B) 归并排序 (D) 堆排序

6. 在文件局部有序或文件长度较小的情况下，最佳的排序方法是 _____。

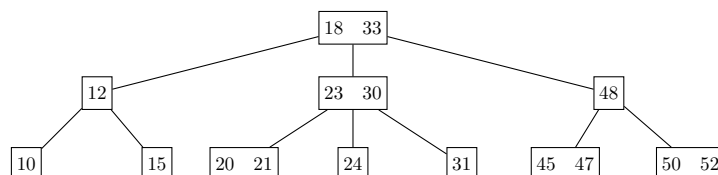
- (A) 直接插入排序 (C) 直接选择排序
(B) 冒泡排序 (D) 堆排序

7. 设输入的关键字满足 $K_1 > K_2 > \dots > K_n$ ，缓冲区大小为 m ，用置换选择排序方法可产生 _____ 个初始归并段。

8. 设有 13 个初始归并段，其长度分别为 28, 16, 37, 42, 5, 9, 13, 14, 20, 17, 30, 12, 18。试计算最佳 4 路归并树的带权路径长度 $WPL =$ _____。
9. 设有序顺序表中的元素依次为 017, 094, 154, 170, 275, 503, 509, 512, 553, 612, 677, 765, 897, 908，则对其进行折半查找时，等概率下搜索成功的平均查找长度和不成功的平均查找长度分别为 _____ 和 _____。
10. 设散列表长度为 14(地址下标为 $0 \cdots 13$)，哈希函数为 $H(key) = key \% 11$ ，表中已有数据的关键字为 15, 38, 61, 84 共四个，现要将关键字为 49 的结点加入表中，用二次探查法解决冲突，则放入的位置是 _____。
- (A) 8 (C) 5
(B) 3 (D) 9
11. 在一棵含有 1023 个关键字的 4 阶 B 树中进行查找 (不读取数据主文件)，至多读盘 _____ 次。
- (A) 7 (C) 10
(B) 8 (D) 9
12. 在一棵阶为 k 的红黑树中，内部结点最少有 $2k-1$ 个；从根到叶的简单路径长度的范围为 $[k, 2k]$ 。

二、辨析与简答 (30 分)

13. (8 分) 将关键字序列 (7、8、30、11、18、9、14) 散列存储到散列表中。散列表是一个下标从 0 开始的一维数组，散列函数为： $H(key) = (key * 3) \text{ MOD } 7$ ，处理冲突采用线性探测法，要求装填 (载) 因子为 0.7。(1) 请画出所构造的散列表。(2) 分别计算等概率情况下查找成功和查找不成功的平均查找长度。
14. (6 分) 设有如下一棵 3 阶 B 树，请画出在其中插入关键码 19 后的 B 树，并给出



插入过程中的访外 (读写) 次数；在此基础上再删除关键码 48，请画出删除后的 B 树及删除过程中的访外 (读写) 次数。

15. (6 分) 将下列字符序列 E A S Y Q U E S T I O N 依次插入到初始为空的红黑树 (RB-tree) 中，请画出最终得到的红黑树。(用圆圈表示红色结点，方框表示黑色结点，外部空叶结点可省略不画)。
16. (4 分) 利用广义表的 Head 和 Tail 运算，把原子 d 分别从下列广义表中分离出来， $L1 = (((((a), b), d), e))$;
 $L2 = (a, (b, ((d)), e))$ 。
17. (6 分) 将关键码 1, 2, 3, ..., $(2k-1)$ 依次插入到一棵初始为空的 AVL 树中，试证明结果是一棵高度为 k 的完全满二叉树。

三、算法填空 (18 分) 阅读和完成下面的代码 (每空可不限一条语句)。

18. (10 分) 以单向链表为存储结构，实现选择排序算法 (得到非递减序列)。

```
class Node:
    def __init__(self, key, next=None):
        self.key = key
        self.next = next
```

```
def sort_linked_list(head):
    i = head
    while i is not None:
        # 填空 1
        while j is not None:
            if ____:          # 填空 2
                # 填空 3
            # 填空 4
        # 填空 5
    return head
```

19. (8 分) 下面是败者树在内部结点从右分支向上比赛的成员函数实现，请将空缺部分补充完整。

```
def play(p, lc, rc, winner, loser):
    B[p] = loser(L, lc, rc)
    # 填空 1
    while p > 1 and p % 2 == 1:
        temp2 = winner(L, temp1, B[p // 2])
        # 填空 3
        temp1 = temp2
        p //= 2
    # 填空 4
```

20. (8 分) 下述算法实现在图 G 中计算从顶点 i 到顶点 j 之间长度为 len 的简单路径条数。图的 ADT 如下：

```
class Graph:
    def first_edge(self, vertex): ...
    def next_edge(self, edge): ...
    def is_edge(self, edge): ...
    def to_vertex(self, edge): ...

visited = [False] * MAXSIZE

def get_path_num_len(G, i, j, length):
    if ____:          # 填空 1
        return 1
    total = 0          # total 表示通过本结点的路径数
    visited[i] = True
    e = G.first_edge(i)
    while G.is_edge(e):
        v = G.to_vertex(e)
        if not visited[v]:
            # 填空 2
        e = G.next_edge(e)
    visited[i] = False  # 允许该结点出现在另一条路径中
    return total
```

四、算法设计与分析 (25 分) 请尽量按照试题中的要求来写高效率的可读算法。应该申明算法思想，在代码中加以恰当的注释。

21. 伸展树是一种自平衡的 BST 树，它可以通过旋转来实现树结构的动态调整。伸展树结点的数据结构定义如下：

```
class TreeNode:
    def __init__(self, data):
        self.data = data
        self.father = None
        self.left = None
        self.right = None
```

其成员函数包括：1)Delete(x): 删除以 x 结点为根的子树；2)Splay(x, f): 将 x 旋转为 f 的子结点 (f 是 x 的祖先)。特殊地，把 x 旋转到根结点即 Splay(x, None)。请运用上述给定的成员函数，实现删除树 rt 中所有大于 u 小于 v 的结点 (假设 u, v 是 Splay 树中的结点，且树中所有结点值不重复)。函数原型为：delete_between($root, u, v$)。解答：把 u 结点旋转到根；把 v 旋转为 u 的右儿子；删除 v 结点的左子树前面几句各 3 分，最后一句 1 分。如果思路、注释各 1 分，不写则扣。

```
def delete_between(root, u, v):
    Splay(u, None)
    Splay(v, u)
    Delete(v.left)
    v.left = None
    return u
```

数据结构与算法 A

common-15 秋-期末

一、选择题 (每题 2 分, 共 20 分)

- 若要一个运行时间为 $100n^2$ 的算法在相同机器上快于另一个运行时间为 2^n 的算法, 则最小的 n 值应为 _____。
- 某算法仅含顺序执行的语句 1 和语句 2, 语句 1 执行次数为 $20n^2 + 3n \log n$, 而语句 2 执行次数为 $2n^3$, 则该算法的时间复杂度为 _____。
- 下面术语中, _____与数据结构的存储结构无关。
(A) 顺序表 (B) 链表 (C) 队列 (D) 循环链表 (E) 堆
- 在一个具有 n 个结点的有序单链表中插入一个新结点并仍保持其有序的时间复杂度为 _____。
(A) $O(n \log_2 n)$ (C) $O(n)$
(B) $O(1)$ (D) $O(n^2)$
- 若元素 a, b, c, d, e, f 依次进栈, 允许进栈、退栈操作交替进行, 但不允许连续三次进行退栈操作, 则不可能得到的出栈序列是 _____。
(A) dcebfa (C) bcaefd
(B) cbdaef (D) afedcb
- 表达式 $3 * 2^{(4 + 2 * 2 - 6 * 3)} - 5$ 的求值过程中, 当扫描到 6 时, 操作数栈和运算符栈为 _____, 其中 $\wedge$ 为乘幂。
(A) 3, 2, 4, 1, 1; $(\wedge(+ * -$ (C) 3, 2, 4, 2, 2; $(\wedge(-$
(B) 3, 2, 8; $(\wedge-$ (D) 3, 2, 8; $(\wedge(-$
- 有 4 棵结点存储整数关键码的二叉树, 若它们的中序遍历序列分别为: A. 5, 3, 1, 2, 4; B. 1, 2, 3, 4, 5; C. 5, 4, 3, 2, 1; D. 1, 3, 5, 4, 2。请问其中可能是二叉搜索树的包括 _____。
- (4 分) 一棵 Huffman 树的带权外部路径长度定义为所有叶结点权值与其深度的乘积之和。设根结点的深度为 0, 对集合 $\{2, 3, 5, 7, 8\}$ 构造二叉 Huffman 树, 则其最小带权外部路径长度为 _____。
- 两个字符串相等的充分必要条件是 _____。(两字符串的长度相等且两字符串中对应位置的字符也相等)
- 2-3 树是一种满足以下两个条件的特殊的树: (1) 每个内部结点有两个或三个子结点; (2) 所有叶结点到根的路径长度相同。如果一棵 2-3 树有 9 个叶结点, 那么它可能有 _____个非叶结点。

二、辨析与简答题 (共 36 分)

- (6 分) 什么是循环队列? 循环队列的优点是什么? 如何判别它的空和满?
- (6 分) 在包含 n 个结点的单链表、双链表和单循环链表中, 若仅知道指针 p 指向某结点, 不知道表的头指针, 能否将结点 p 从相应的链表中删去? 若可以, 其时间复杂度各为多少?
- (8 分) 请分别计算模式 $p = \text{"aabaac"}$ 优化前和优化后的 next 数组, 并按照优化后的 next 数组针对目标 $t = \text{"aabaabaabaac"}$ 进行 KMP 快速模式匹配, 请画出匹配过程的示意图并计算匹配过程中的比较次数。
- (8 分) 根据树的双标记层次遍历序列, 求其带度数的后根遍历序列。譬如, 已知一棵树的双标记层次遍历序列如下:

```
A : ltag: 0 , rtag 1 B : ltag: 0 , rtag 0 C : ltag: 0 , rtag 1 D : ltag: 1 , rtag 0 G :
ltag: 0 , rtag 1 E : ltag: 0 , rtag 1 H : ltag: 1 , rtag 1 F : ltag: 1 , rtag 0 I : ltag:
1 , rtag 1
```

则其带度数的后根遍历序列为：D0 H0 G1 B2 F0 I0 E2 C1 A2(注：各个结点按照” 结点名度数” 的方式给出，结点之间用空格分隔)。现某棵树的双标记层次遍历序列如下：

```
A : ltag: 0 , rtag 1 B : ltag: 0 , rtag 0 G : ltag: 1 , rtag 1 C : ltag: 0 , rtag 0 F :
ltag: 0 , rtag 1 D : ltag: 1 , rtag 0 E : ltag: 1 , rtag 1 H : ltag: 1 , rtag 0 I : ltag:
1 , rtag 1
```

请给出其带度数的后根遍历序列。

15. (8 分) 使用重量权衡合并规则 (权重合并规则) 与路径压缩，对下列从 0 到 15 之间的数的等价对进行归并。在初始情况下，集合中的每个元素分别在独立的等价类中。当两棵树规模同样大时，使结点数值较大的根结点作为值较小的根结点的子结点。

(0, 2) (1, 2) (3, 4) (3, 1) (3, 5) (9, 11) (12, 14) (3, 9) (4, 14) (6, 7) (8, 10) (8, 7) (7, 0) (10, 15) (10, 13)

请填写下面表格的空白部分树的父指针表示法的数组表示。也就是所有等价对都被处理之后，所得父结点的下标值 (没有父结点则填”-1”)。

父结天下标

结点值

0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15

结点的下标 0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15

三、算法填空题 (每题 12 分，共 24 分)

16. (每空 3 分，共 12 分) 下面的算法利用一个栈将一给定的序列从小到大排序。长度为 n 的序列由 $1, 2, \dots, n$ 这 n 个连续的正整数组成。请补全下面的代码段，使其可以判断给定的序列是否可以利用一个栈进行排序，如果可以，输出相应的操作。

例如：序列 4 3 1 2，此序列可以排序，输出：push push push pop push pop pop pop。

```
class Stack:
    def clear(self): ...
    def push(self, item): ...
    def pop(self): ...
    def top(self): ...
    def is_empty(self): ...
    def is_full(self): ...

s = Stack()                # s 为栈
sequence = [...]           # 待排序序列
n = len(sequence)

def is_legal():             # 判断序列是否可以排序
    for i in range(n):
        for j in range(i + 1, n):
            for k in range(j + 1, n):
                if ____:    # 填空 1
                    return False
    return True

def transform():            # 输出转换操作
    cur, i = 1, 0
```

```

while ____:          # 填空 2
    while s.is_empty() or ____:    # 填空 3
        ____          # 填空 4
        print("push", end=" ")
        if cur == s.top():
            break
    s.pop()
    print("pop", end=" ")
    cur += 1

```

17. (每空 3 分，分析 3 分，共 12 分) 以下算法用来判断两棵树是否相同，试补全下面的代码段，并分析算法的运行时间代价。

```

def is_equal(root1, root2):
    while root1 is not None and root2 is not None:
        if root1.value() != root2.value():
            return ____          # 填空 1
        if ____: # 填空 2
            return False
        root1 = root1.right_sibling()
        root2 = root2.right_sibling()
    if ____: # 填空 3
        return True
    return False

```

四、分析证明题 (共 20 分)

18. (5 分) 请证明非空满 K 叉树的叶结点数目为 $(K - 1) * n + 1$ ，其中 n 为分支结点数目。
19. (15 分) 有 n 个数 $K_1, K_2, \dots, K_n$ 顺序存储在数组中，形成一棵 n 个结点的完全二叉树。现在要将它按如下方式建成一个最小值堆：从左至右扫描整个数组，将第 i 个元素 K_i ($i = 1, 2, \dots, n$) 与其父结点 $K_{\lfloor i/2 \rfloor}$ 比较，如果 $K_i \geq K_{\lfloor i/2 \rfloor}$ ，则不再进行操作；若 $K_i < K_{\lfloor i/2 \rfloor}$ ，将 2 个结点对换，再将 $K_{\lfloor i/2 \rfloor}$ 与 $K_{\lfloor i/4 \rfloor}$ 比较，以此类推，直到比较到根结点 K_1 。
- (1) 按照这种建堆方式，最坏情况下建堆的时间复杂度是多少？
 - (2) 教材上使用的“筛选法”建堆 (Sift-down)，最坏情况下建堆的时间复杂度是多少？
 - (3) 上述 2 种建堆法有类似之处，请问它们的时间复杂度是否一样，若不同，则请说明导致它们时间复杂度不同的原因。

数据结构与算法 A

common-16 秋-期中

一、选择题 (每题 2 分, 共 20 分)

1. 下面函数的时间复杂度是_____。

```
def work(n, m):
    total = 0
    i = 1
    while i <= m:
        j = 1
        while j <= n:
            total += i * j
            j *= 2
        i *= 2
    return total
```

2. 下面关于线性表的叙述中, 正确的是哪些?

- (A) 采用顺序存储的线性表, 必须占用一片连续的存储单元。
- (B) 采用顺序存储的线性表, 便于进行插入和删除操作。
- (C) 采用链接存储的线性表, 不必占用一片连续的存储单元。
- (D) 采用链接存储的线性表, 便于插入和删除操作。
- (E) 采用链接存储的线性表, 插入第 i 个元素的时间与 i 的数值成正比。
- (F) 采用链接存储的线性表, 在已知的结点 p 后插入一个新节点的时间复杂度与结点 p 的位置有关。

3. 若元素 a、b、c、d 依次进栈, 则可能的出栈序列有_____种; 若有元素 a、b、c、d、e 依次进栈, 则可能的出栈序列有_____种。

4. 一个字符串中_____称为该串的子串。abcaab 的不相等真子串个数为_____。

5. 一个具有 n 个结点的二叉树的叶结点的个数最多为_____。

6. 一个有 5 层结点的完全 3 叉树。按前序遍历周游给结点从 1 开始编号, 则第 100 号结点的父结点的编码为_____。(注释: 独根树为 1 层的)

7. 将序列 {12, 70, 33, 65, 24, 56, 48, 92, 86, 33} 按建堆方法调整为小值堆, 调整后的序列为_____。

8. 假设用于通信的电文仅由 8 个字符 {a,b,c,d,e,f,g,h} 组成, 字符在电文中出现的频率分别为 0.07, 0.19, 0.02, 0.06, 0.32, 0.03, 0.21, 0.10, 对这些字符进行哈夫曼编码的外部路径长度为_____。

9. 将森林 F 转换为对应的二叉树 T, F 中的叶结点的个数为_____。

- (A) T 中叶结点的个数
- (B) T 中度为 1 的结点个数
- (C) T 中左子指针为空的结点个数
- (D) T 中右子指针为空的结点个数

10. 一棵树 T 有 20 个度为 4 的结点, 10 个度为 3 的结点, 1 个度为 2 的结点, 10 个度为 1 的结点, 则树 T 的结点个数为_____。

二、辨析与简答 (共 32 分)

11. (6 分) 请谈谈循环队列和链表的优缺点。当遇到如下情况时, 应当使用哪一种数据结构?

情况：你要维护食堂鸡腿饭的排队队伍，队伍有两种操作：1. 排在队首的人出来打鸡腿饭后离开；2. 一个新来的人排在队尾。你知道食堂不太大，所以队伍的最大长度你可以估计出来。

12. (4 分) 如何找出具有相同的中序序列和后序序列的所有二叉树？
13. (4 分) 从空二叉树开始，将关键码 {18,73,10,5,68,99,27,41,51,32,25} 严格按照 BST(二叉搜索树) 的插入算法逐个插入 BST，画出这样构造出的 BST 树，并写出对该 BST 按照前序遍历得到的序列。
14. (6 分) 以下是计算模式 P 的 next 向量的算法：

```
def find_next(P):
    i, k = 0, -1
    m = len(P)
    nxt = [0] * m
    nxt[0] = -1
    while i < m:
        while k >= 0 and P[i] != P[k]:
            k = nxt[k]
        i += 1
        k += 1
        if i == m:
            break
        if P[i] == P[k]:
            nxt[i] = nxt[k]
        else:
            nxt[i] = k
    return nxt
```

采用上述算法，计算模式 P="abcdaabcb" 的 next 向量。

i
P[i]
a
b
c
d
a
a
b
c
a
b
next[i]

15. (6 分) 数组 {23, 17, 14, 6, 13, 10, 1, 5, 8, 12} 是否为一个最大值堆？若是，请说明理由，否则请严格按照筛选法建堆的过程将其调整成为最大值堆，并画出逐步调整建堆的过程。
16. (6 分) 对下列 15 个等价对进行合并，给出所得等价类树的图示。在初始情况下，集合中的每个元素分别在独立的等价类中。使用重量权衡合并规则，合并时子树结点少的并入结点数多的那棵 (多的那个作为新树根，少的那个根作为新根的直接子结点)；若两棵树规模同样大，则把根值较大的并入根值较小 (新树根取值小的)。同时，采用路径压缩优化。(0,2) (1,2) (3,4) (3,1) (3,5) (9,11) (12,14) (3,9) (4,14) (6,7) (8,10) (8,7) (7,0) (10,15) (10,13)

三、算法填空 (每空 2 分，共 16 分)

17. 1. 下面的算法将一个用带度数的后根次序法表示的森林转换为左子结点/右兄弟结点法表示。请利用题目给出的树结点 ADT 和栈 ADT，填充空格，使其成为完整的算法。空格中可能需要填写 0 到多条语句 (或表达式)。

```

class Stack:
    def empty(self): ...
    def size(self): ...
    def top(self): ...
    def push(self, x): ...
    def pop(self): ...

class Node:
    def __init__(self, info, degree):
        self.info = info
        self.degree = degree

class TreeNode:
    def __init__(self, value):
        self.value = value
        self.child = None
        self.sibling = None

    def set_child(self, pointer):
        self.child = pointer

    def set_sibling(self, pointer):
        self.sibling = pointer

def convert(nodes):
    cstack = Stack()
    for i in range(len(nodes)):
        cur = TreeNode(nodes[i].info)
        if nodes[i].degree == 0:
            # 填空 1
        else:
            assert nodes[i].degree <= cstack.size()
            temp2 = None
            for j in range(_____):      # 填空 2
                temp1 = cstack.top()
                cstack.pop()
                # 填空 3
                temp2 = temp1
            # 填空 4
            cstack.push(cur)

        cur = temp2 = None
        while not cstack.empty():
            cur = cstack.top()
            cstack.pop()
            # 填空 5
            temp2 = cur
    return cur

```

18. 2. 给定一个二叉树的根结点和树中任意两个结点，补全下面的程序段以求这两个结点的最近公共祖先。

```

def lowest_common_ancestor(root, p, q):
    if _____:      # 填空 6
        return root
    left = lowest_common_ancestor(root.left, p, q)

```

```
right = lowest_common_ancestor(root.right, p, q)
if ____:                                # 填空 7
    return root
return left if ____ else right         # 填空 8
```

四、算法设计与实现 (16 分)

19. (8 分) 现有两个满足先进先出原则的队列 A 和 B，在队列上可进行以下 4 个操作：front(): 获得队首元素；push_back(T x): 把元素 x 插到队尾；pop_front(): 删除队首元素；empty(): 判断是否为空。请用 A 和 B 模拟一个栈的 4 个操作：top(), pop(), push(), empty()。只要给出实现栈操作的基本思想，对时间效率无具体要求。
20. (8 分) 对于两个串 A 和 B，给出一种算法使得能够保证正确地求出最大的 L 值，使得 A 串中长为 L 的前缀与 B 串中长为 L 的后缀相等。

五、分析证明题 (共 16 分)

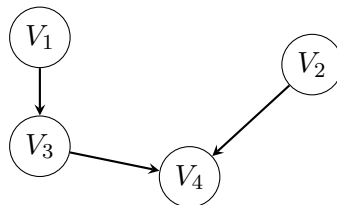
21. (6 分) 试证明在一棵树中，u 是 v 的祖先，当且仅当在先序遍历中 u 在 v 之前且在后序遍历中 u 在 v 之后。
22. (10 分) 对于满二叉树：
- (1) 试问具有 n 个叶结点的树中共有多少个结点？
 - (2) 试证明：
- ，其中 n 为叶结点的个数，il 表示第 i 个叶结点所在的层次 (设根结点所在的层次为 0)。

数据结构与算法 A

common-16 秋-期末

一、选择填空题 (每空 1 分, 共 11 分)

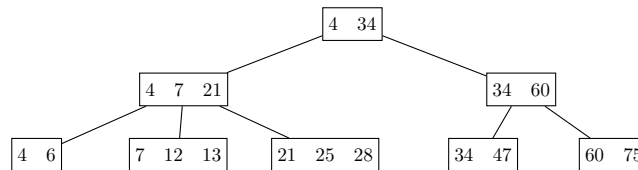
1. G 是一个非连通无向图, 共有 21 条边, 则图 G 至少有 8 个顶点。
2. 对于一个包含 $N(N>1)$ 个顶点的图, 假定任意两点间最多只有一条边, 那么下列哪些情况是错误的 _____。
 (A) 如果是有向图, 则其任何一个极大强连通子图都无法进行拓扑排序。
 (B) 如果是无向连通图, 则其最小生成树一定不包括权重最大的边。
 (C) 如果是无向连通图, 假设所有边的权重均为正值, Dijkstra 算法给出的生成树不一定是最小生成树, 但是与该图的任何一个最小生成树都至少有一条相同边。
3. 有向图 G 如下图所示:



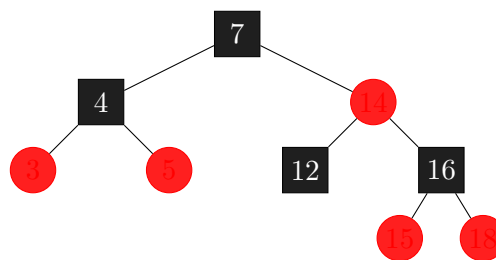
- (1) 写出所有可能的拓扑序列: _____。
- (2) 若要使该图只有唯一的拓扑序列, 则可以添加一条弧 _____。
4. 在快速排序中, 定义一次平分的划分为“幸运的划分”, 而一次划分如果有一边为空则是“不幸的划分”。假设划分的过程总是“幸运”和“不幸”交替的, 则该快速排序的时间复杂性为 $n \log n$ 。
5. 具有 10000 个关键码的 16 阶 B 树的查找路径长度 (从根到叶节点访问 B 树索引块的次数) 不会小于 3。
6. 设有 8 个初始归并段, 其长度分别为 32, 46, 56, 64, 20, 87, 70, 40; 进行 3 路归并排序, 所构造的最佳归并树对应的总读写次数为 1590。
7. $A[N][N]$ 是对称矩阵, 现将下三角矩阵按行存储到一维数组 $T[N(N+1)/2]$ 中 (包括对角线), 则对任一上三角元素 $A[i][j]$ 其对应值 ($0 \leq i \leq j < N$) 在 $T[k]$ 中的下标 k 是 $j(j+1)/2+i$ 。
8. 在一棵空 AVL 树中, 顺序插入如下关键码: {5, 9, 4, 2, 1, 3, 8}, 请问全部插入后, 在等概率下查找成功的平均检索长度为 $17/7$ 。
9. 已知广义表 $C=(c, (d, A), B, e)$, 则广义表 C 的深度为 2, $\text{tail}(\text{head}(\text{tail}(C)))$ 的运算结果为 (A)。

二、简答辨析题 (每题 3 分, 共 15 分)

10. 如果要找出一个具有 n 个元素集合中的第 k ($1 \leq k \leq n$) 个最小元素，所学过的排序方法中哪种最适合？给出实现的基本思想。
11. 已知一组关键码为 (26, 36, 41, 38, 44, 15, 68, 12, 06, 51, 25)，散列表长度为 15，用线性探查法解决冲突构造这组关键码的散列表。散列函数为 $h(k) = k \bmod 13$ 。请回答：
- (1) 构造顺序插入上述关键码集合后的散列表；
 - (2) 下一记录放到第 11 个槽和第 7 个槽中的概率分别是多少？
 - (3) 查找成功和失败情形下的平均查找长度 ASL 分别是多少？
12. 有如下图所示的一个 3 阶 B+ 树，请分别画出依次插入关键码 10 和删除关键码 4 的 B+ 树，并分析上述操作过程中的访外读写次数。



13. 一棵红黑树如下图所示 (空树叶未画出)，请首先画出插入节点 17 的红黑树，在此基础上，然后画出删除节点 14 的红黑树。(画图说明：黑节点用方框，红节点用圆圈，不画空树叶。)



14. 字符串的后缀是指字符串的任意尾部串。比如“abbc”的后缀有“abbc”，“bbc”，“bc”，“c”和\$ (表示为空串)。现有两个字符串“window”和“indigo”，请画出它们所有后缀串所组成的 Trie 树 (注意：对路径进行压缩，并把局部子串标注在相应的边上)。

三、算法填空题 (每空 1 分，共 5 分)

15. 下述算法实现在图 G 中计算从顶点 i 到顶点 j 之间长度为 len 的简单路径条数。图的 ADT 如下：

```
class Graph:
    def first_edge(self, vertex): ...
    def next_edge(self, edge): ...
    def is_edge(self, edge): ...
    def to_vertex(self, edge): ...

visited = [False] * MAXSIZE

def get_path_num_len(G, i, j, length):
    if i == j and length == 0:
        return 1

    total = 0
    visited[i] = True
    e = G.first_edge(i)
    while G.is_edge(e):
        v = G.to_vertex(e)
```

```

    if not visited[v]:
        total += get_path_num_len(G, v, j, length - 1)
        e = G.next_edge(e)

    visited[i] = False
    return total

```

16. 下面的代码实现了一种计数排序：对每个待排序记录，扫描整个序列统计比它小的记录个数 `count`，`count` 即是该记录在序列中的争取位置。请将下面是计数排序的程序代码补充完整。

```

def sort_records(array):
    cur_index = 0          # 待排下标
    n = len(array)
    while cur_index < n:
        count = 0
        for j in range(n):    # 统计比 array[cur_index] 小的值的个数
            if array[j] < array[cur_index]:
                count += 1

        array[cur_index], array[count] = array[count], array[cur_index]
        if cur_index == count:
            cur_index += 1
    return array

```

四、设计分析题 (共 9 分)

17. (3 分) 某小区有 N 座别墅需要供水。在第 i 座别墅里挖井需要 $w[i]$ 的费用，在第 i 座和第 j 座别墅之间铺水管需要 $c[i][j]$ 的费用。给每座别墅供水，要么挖井、要么跟其他有井的别墅铺设连通的水管路径。请设计算法，求解使每座别墅都得到供水的最小费用方案。
18. (6 分) 现有一个工资系统，请实现满足如下操作需求的 Splay 树，并给出算法的伪代码：(Splay 树根为 `root`，其他变量可以自己定义)
- 1) `insert(w)`: 新加一位工资为 w 的员工 (假设 w 没有重复值);
 - 2) `fire(t)`: 解雇工资大于 t 的员工。
- 相关定义和函数说明如下：
- 伸展树是一种自平衡的 BST，数据结构如下：

```

class TreeNode:
    def __init__(self, wage):
        self.wage = wage
        self.father = None
        self.left = None
        self.right = None

```

可以直接使用的函数：

```

def Splay(x, f):
    """ 将 x 旋转为 f 的子结点；f 为 None 时把 x 旋到根。"""

def find(x, subtree):
    """ 在 subtree 中查找 x；若失败，返回 x 应插入位置的父结点。"""

```

```
def Delete(x):
    """ 删除以 x 为根的子树。"""

    root = None

def insert(w):
    global root
    if root is None:
        root = TreeNode(w)
        return

    parent = find(w, root)
    new_node = TreeNode(w)
    new_node.father = parent
    if w < parent.wage:
        parent.left = new_node
    else:
        parent.right = new_node
    Splay(new_node, None)
    root = new_node

def fire(t):
    global root
    if root is None:
        return

    boundary = find(t, root)
    if boundary.wage == t:
        Splay(boundary, None)
        Delete(boundary.right)
        boundary.right = None
        root = boundary
        return

    parent = boundary
    sentinel = TreeNode(t)
    sentinel.father = parent
    if t < parent.wage:
        parent.left = sentinel
    else:
        parent.right = sentinel

    Splay(sentinel, None)
    Delete(sentinel.right)
    root = sentinel.left
    if root is not None:
        root.father = None
```

数据结构与算法 A

common-17 秋-期中

一、选择与填空 (每空 2 分, 共 24 分)

1. 下面函数的时间复杂度是 ()

```
def recursive(n, m, k):
    if n <= 0:
        print(m, k)
    else:
        recursive(n - 1, m + 1, k)
        recursive(n - 1, m, k + 1)
```

- (A) $O(n * m * k)$ (C) $O(2^n)$
 (B) $O(n^2 * m^2)$ (D) $O(n!)$

2. 完成在双循环链表结点 p 之后插入 s 的操作为 ():

- (A) p.next.prev=s; s.prev=p; s.next=p.next; (C) s.next=p.next; s.prev=p; p.next.prev=s;
 p.next=s; p.next=s; s.prev=p; (D) s.prev=p; s.next=p.next; s.prev.next=s;
 s.next=p.next; s.next.prev=s;

3. 设栈 S 和队列 Q 初始状态为空, 元素 e1, e2, e3, e4, e5, e6 依次通过栈 S, 一个元素出栈后即进队列 Q, 若 6 个元素出队序列是 e2, e4, e3, e6, e5, e1, 则栈 S 的容量至少是 ()

- (A) 2; (C) 4;
 (B) 3; (D) 6

4. 设循环队列的容量为 40 (序号从 0 到 39), 现经过一系列的入队和出队运算后: 1) front=12, rear=19; 2)front=19, rear=12; 在这两种情况下, 循环队列中的元素个数分别为 _____和 _____。

5. 一个 n 位的字符串共有 _____个子串。

6. 已知二叉树有 n 个结点 ($n > 0$), 则度为 2 的结点最多有 _____个。

7. 用数组存储一个有 50 个元素的最小值堆。已知堆中没有重复元素。给出最大元素的下标可能的取值范围为 _____到 _____。

8. 假设用于通信的电文由 5 个字符组成。已知其中一个字符的 Huffman 编码为 1101, 则该 Huffman 编码树的平均编码长度为 _____。

9. 假设先根次序遍历某棵树的结点次序为 GABCDIJEFH, 后根次序遍历该树的结点次序为 BADIJCFEHG。那么 I 结点的父结点是 _____。
10. 一个拥有 144 个结点的完全三叉树, 现在从倒数第二层上任取一个结点, 该结点为叶结点的概率为 _____。

二、辨析与简答 (共 24 分)

11. (6 分) 现有一个由单链表和循环单链表构成的特殊链表, 单链表的头元素是 head, 末尾元素为 tail, head...tail 即是一条普通的链表, 同时 tail 元素还是另一条循环单链表的头元素。如 A.B.C.D.E.F.G.H.E 就是一个特殊链表, 其中 head 为 A, tail 为 E。
现在你只知道 head, 但是 you 想知道这个循环链表中有多少元素, 但同时你的剩余空间十分的小, 所以你想用尽可能小的空间来解决问题, 请问你会怎么做? (依据占用空间给分)
12. (6 分) 假设以 I 和 O 分别表示入栈和出栈操作, 栈的初始和终态均为空, 入栈和出栈的操作序列可表示为仅由 I 和 O 组成的序列, 称可以操作的序列为合法序列, 否则为非法序列。如 IOIOIOIO 合法, IOOIOIO 和 IIOIOIO 为非法。请给出一个算法, 判断所给操作序列是否合法。
13. (6 分) 给定一棵用数组存储的完全二叉树 {10, 5, 12, 3, 2, 1, 8}
(1) 用筛选建堆法将该完全二叉树调整为最大值堆。画出调整后得到的最大值堆, 并说明在调整过程中都依次交换了哪些元素。(注: 画堆只需要写出层次遍历序列结果即可)
(2) 将 6、7、14 按顺序插入 (1) 得到的最大值堆, 堆的空间足够大, 画出插入 14 后得到的最大值堆, 并说明在插入 14 的过程中都依次交换了哪些元素。
(3) 对 (2) 得到的堆进行 3 次 deleteMax, 画出第 3 次 deleteMax 后得到的堆, 并说明在第 3 次 deleteMax 的过程中都依次交换了哪些元素。
14. (6 分) 对下列 15 个等价对进行合并, 给出所得等价类树的图示。在初始情况下, 集合中的每个元素分别在独立的等价类中。使用重量权衡合并规则, 合并时子树结点少的并入结点数多的那棵 (多的那个作为新树根, 少的那个根作为新根的直接子结点); 若两棵树规模同样大, 则把根值较大的并入根值较小 (新树根取值小的)。同时, 采用路径压缩优化。
(0,2) (1,2) (3,4) (3,1) (3,5) (9,11) (12,14) (12,9) (4,14) (6,7) (8,10) (8,7) (7,11) (10,15) (10,13)

三、算法填空 (每空 3 分, 共 24 分)

15. 下面的算法利用一个栈将一个给定的序列从小到大排序。长度为 n 的序列由 1, 2, ..., n 这 n 个连续的正整数组成。请补全下面的代码段, 使其可以判断给定的序列是否可以利用一个栈进行排序, 如果可以, 输出相应的操作。
例如: 序列 4 3 1 2, 此序列可以排序, 输出: push push push pop push pop pop pop。

```
class Stack:
    def clear(self): ...
    def push(self, item): ...
    def pop(self): ...
    def top(self): ...
    def is_empty(self): ...
    def is_full(self): ...
```

```
s = Stack()
sequence = [...]
n = len(sequence)
```

```
def is_legal():
    # 判断序列是否可以排序
```

```

    for i in range(n):
        for j in range(i + 1, n):
            for k in range(j + 1, n):
                if ____: # 填空 1
                    return False
    return True

def transform():
    # 输出转换操作
    cur, i = 1, 0
    while ____: # 填空 2
        while s.is_empty() or ____: # 填空 3
            ____ # 填空 4
            print("push", end=" ")
            if cur == s.top():
                break
        s.pop()
        print("pop", end=" ")
        cur += 1

```

16. 下面的算法将一个用带右链的先根次序法表示的森林转换为用带度数的后根次序法表示。请利用题目给出的树结点 ADT 和栈 ADT，填充空格，使其成为完整的算法。

```

class RlinkTreeNode:
    def __init__(self, info, ltag, rlink=None):
        self.info = info
        self.ltag = ltag # 1 表示没有左子结点
        self.rlink = rlink # 右兄弟

class PostTreeNode:
    def __init__(self):
        self.info = None
        self.degree = 0

post_tree = [PostTreeNode() for _ in range(n)]
count = 0

def convert(node):
    """ 返回右兄弟链上的结点数量 (包括自己)。"""
    global count
    if node.ltag == 1:
        # 填空 5
        post_tree[count].degree = 0
        count += 1
    else:
        tmp = convert(next_node(node)) # 访问第一个子结点
        post_tree[count].info = node.info
        # 填空 6
        count += 1

    if ____: # 填空 7
        return convert(node.rlink) + 1
    else:
        # 填空 8

```

四、算法设计与实现 (14 分)

17. (8 分) 编写算法伪码, 对给定的字符串 `str`, 返回其最长重复子串及其下标位置。如 `str="abcdaccdac"`, 则子串"cdac" 是 `str` 的最长重复子串, 下标为 2。
(重复子串允许重叠)
18. (6 分) 给出一个算法, 求一棵树的树高。可以写出伪代码, 需要相应的注释, 或者写出详细算法思路。对时间复杂度无具体要求。

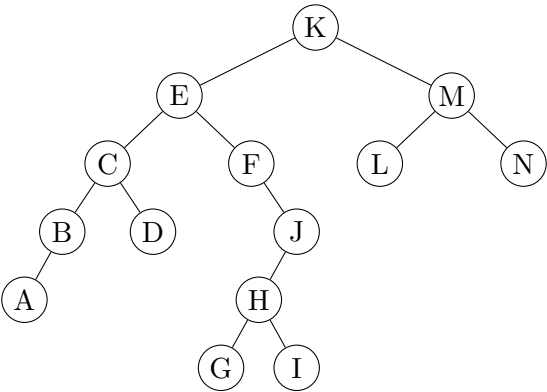
五、分析证明题 (共 14 分)

19. (8 分) 二叉树的内部路径长度是指所有节点的深度的和。假设将 N 个互不相同的随机元素插入一棵空的二叉搜索树。请证明得到的二叉搜索树的内部路径长度期望为 $O(N\log N)$ 。
20. (6 分) 证明按先根次序周游树获得的序列与其对应二叉树的前序周游获得的序列相同。

数据结构与算法 A
common-17 秋-期末

一、选择填空题 (7 小题, 每空 1 分, 共 10 分)

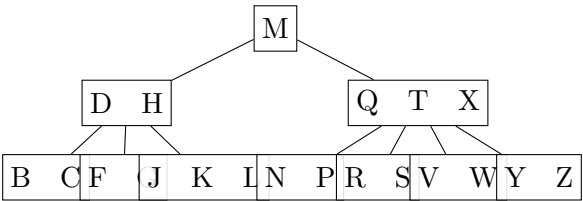
1. 下列说法中正确的是 _____。
- (A) 图的广度优先搜索中邻接点的寻找具有”先进后出”的特征。
(B) 连通图的生成树是该图的一个极大连通子图。
(C) 图的广度优先搜索是一个递归过程。
(D) 在非连通图的遍历过程中, 每调用一次深度优先搜索算法都得到该图的一个连通分量。
2. 基于比较的内排序算法中, 平均时间复杂度为 $O(n\log n)$ 的算法有 _____, 这其中稳定的排序算法有 _____。
3. 在包含 n 个关键码的线性表中从后向前进行顺序检索 (0 下标为监视哨)。若第 i 个关键码的检索概率为 p_i , 且满足如下分布: $p_1 = 1/2, p_2 = 1/4, \dots, p_n = 1/2^n$, 则成功检索的平均检索长度为 _____, 失败检索的平均检索长度为 _____。
4. 给定一组输入关键码 {15, 1, 3, 14, 11, 2, 12, 10, 4, 17}, 采用大小为 3 的最小堆进行置换选择排序的输出为 _____。
5. 按照 AVL 定义, 下图所示的 BST 中没有达到平衡状态的结点有 _____。



6. 假设磁盘块大小是 1024 字节, 每个索引项需要 8 个字节。一个包含 20000 条记录的数据文件, 若采用二级索引, 则一级索引文件和二级索引文件共需占用 _____ 磁盘块。
7. 给定两个广义表 $S = ((a, (b), c), ((d), e))$, $T = (f, (g, ((h))), i, j)$; 利用广义表的 Head 和 Tail 运算, 则有 $d =$ _____; $h =$ _____。

二、简答辨析题 (5 小题, 共 15 分)

8. (2 分) 将关键码序列 (7, 8, 30, 11, 18, 9, 14) 存储到一个散列表中。散列表是一个下标从 0 开始的一维数组, 若散列函数采用: $H(\text{key}) = (\text{key} * 3) \text{ MOD } 7$, 处理冲突采用线性探测法, 装填 (负载) 因子要求为 0.7。(1) 请画出所构造的散列表; (2) 分别计算等概率情况下查找成功和查找不成功的平均查找长度。
9. (4 分) 下图是一棵关键码为英文字符的 m 阶 B 树, 依据字典序组织。请回答下述问题:

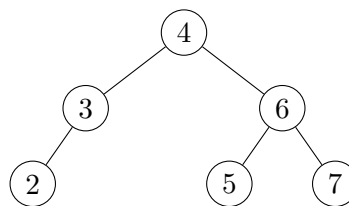


(1) 这是一棵合法的 B 树，则 m 可以取哪些值，并说明原因；(2) 若 m 取所有可能值中的最小值，则 (A) 给出查找字符 V 的过程，并说明需要的读盘次数 (假设根结点也需读盘操作)；(B) 给出插入字符 I ($H < I < J$) 的过程，并画出插入 I 后的 B 树；(C) 在原始 B 树 (即进行第 (B) 问的操作之前) 的基础上，给出删除 H 的过程，并画出删除 H 后的 B 树。

10. (4 分) 现有 8 个顺串，每个顺串的第一个关键码为 $S1[1..8] = \{17, 10, 2, 9, 13, 15, 12, 4\}$ ，第二个关键码是 $S2[1..8] = \{23, 34, 38, 25, 27, 40, 36, 39\}$ ，进行从小到大的归并。(1) 请写出初始构建的赢者树内部结点的数组 $A[1..7]$ 和败者树内部结点的数组 $B[0..7]$ (数组 A 和 B 存储的是 1 到 8 的顺串的下标， $B[0]$ 表示最终冠军)；(2) 在输出第一个全局赢者并重构赢者树或败者树后，写出此时赢者树内部结点的数组 $A[1..7]$ 和败者树内部结点的数组 $B[0..7]$ 。

11. (3 分) 请按照 2, 1, 4, 5, 9 的插入顺序，建立一棵红黑树，并画出每插入一个元素后的红黑树。(用 B 和 R 表示结点的颜色)

12. (2 分) 给定下面的一棵 splay 树，按照结点序列 2, 3, 4, 5, 6, 7 访问所有结点之后，请画出访问后的 splay 树形结构。



三、算法填空题 (2 小题，每空 1 分，共 5 分)

13. 下面是考虑墓碑问题的散列表插入算法，算法中不允许有重复关键码插入。

```

class HashDict:
    EMPTY = object()
    TOMB = object()

    def __init__(self, size):
        self.M = size
        self.currecnt = 0
        self.HT = [self.EMPTY] * size

    def p(self, key, i):
        """ 探查函数。"""

    def h(self, key):
        """ 散列函数。"""

    def get_key(self, elem):
        """ 获取元素 elem 的关键码。"""

    def eq(self, a, b):
        return a == b

    def hash_insert(self, elem):
        inplace = None
        i = 0
        home = self.h(self.get_key(elem))
        pos = home
        tomb_pos = False

        while _____: # 填空 1
            if self.eq(elem, self.HT[pos]):
    
```

```

        return False
    if self.eq(self.TOMB, self.HT[pos]) and not tomb_pos:
        # 填空 2
        -----
        tomb_pos = True
    i += 1
    -----
    # 填空 3: 探查

    if not tomb_pos:
        insplace = pos
        # 若无墓碑, 则插入第一个空槽
        self.HT[insplace] = elem
        self.currnt += 1
    return True

```

14. 设有一个仅由红、白、蓝 3 种颜色的条块组成的序列, 各种色块的个数和分布是随机的, 但 3 种颜色的总数为 n 。下面的代码实现了一种时间复杂度为 $O(n)$ 的排序, 使得这些条块按照红、白、蓝的顺序排好。请将其补全。

```

RED, WHITE, BLUE = 1, 2, 3

def color_sort(array):
    insert_red, insert_white = 0, 0
    for i in range(len(array)):
        if array[i] == WHITE:
            array[i] = array[insert_white]
            array[insert_white] = WHITE
            insert_white += 1

        if array[i] == RED:
            -----
            -----
            array[insert_red] = RED
            insert_red += 1
            insert_white += 1
    return array

```

四、设计分析题 (2 小题, 共 10 分)

15. (6 分) 请设计一个算法求有向无环图 (DAG) 中每对顶点间的”最长简单路径”(所谓”最长简单路径”, 是指该简单路径包含的边数最多)。以一个有向无环图作为输入, 对于每对顶点, 如果它们之间存在简单路径, 则输出其中路径长度最长 (边数最多) 的简单路径, 否则输出空。试分析你所设计算法的时间复杂度。
16. (4 分) 利用证明基于比较的排序问题的复杂度下限的方法, 试证明在已排序序列上基于比较的检索问题的复杂度下限是 $\Omega(\log n)$ 。